

UNIVERSIDAD DE GUADALAJARA

BENEMÉRITA UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Departamento de Ciencias Computacionales

Procesos, Diagramas, Transiciones y Algoritmos de Planificación

5° Actividad de Aprendizaje – Tarea 3

Sistemas Operativos

Código de la materia: IL366
Sección: D09

P R E S E N T A(N)

ICOM - Velázquez Vázquez Valeria - 224785718

PROFESOR(A)

Becerra Velázquez Violeta del Rocío

Guadalajara, Jalisco, Mx.
27 de septiembre del 2026

CUCEI
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

ÍNDICE

ÍNDICE 2
1. Creación y Terminación de Procesos 3
2. Diagrama de 5 Estados 4
3. Algoritmos de Planificación 6
3.a Definición de Algoritmos de Planificación 6
3.b Políticas de Planificación (Apropiativas y No Apropiativas) 6
3.c Tabla Comparativa de Algoritmos 6
4. Elementos de la Tabla de Procesos (BCP) 8
Preguntas de Investigación 9
Captura del Quiz 10
Conclusiones 11
Bibliografía 13

1. Creación y Terminación de Procesos

Un proceso es un programa en ejecución, junto con el conjunto de recursos que el sistema operativo le asigna para llevar a cabo su tarea (memoria, registros, archivos abiertos, etc.). A lo largo de su vida, un proceso pasa por dos momentos clave gestionados directamente por el sistema operativo: su creación y su terminación.

Formas de Creación de Procesos

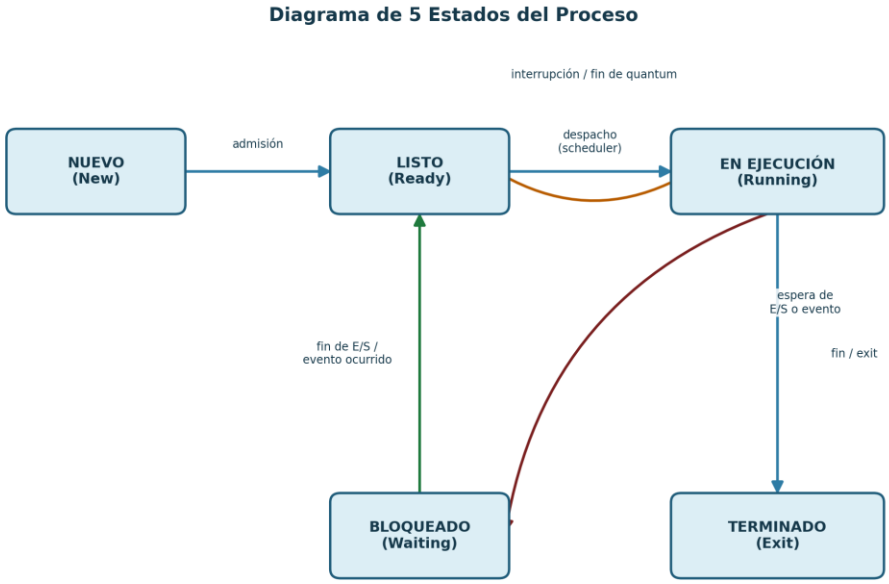
1. *Inicialización del sistema (system initialization): al arrancar el sistema operativo se crean automáticamente los procesos y demonios necesarios para su funcionamiento (procesos en primer y segundo plano), como los encargados de administrar la red, el registro de eventos o la interfaz gráfica.*
2. *Llamada al sistema de creación de procesos por un proceso en ejecución: un proceso (padre) puede crear uno o varios procesos hijos mediante una llamada al sistema, como fork() en sistemas Unix/Linux o CreateProcess() en Windows. Esto permite, por ejemplo, que un shell cree un nuevo proceso para cada comando ejecutado.*
3. *Solicitud de un usuario para crear un nuevo proceso: ocurre cuando el usuario interactúa con el sistema, por ejemplo, al hacer doble clic sobre un ícono, ejecutar un comando en la terminal o abrir una aplicación desde un menú.*
4. *Inicio de un trabajo por lotes (batch job): en sistemas orientados al procesamiento por lotes (mainframes), un proceso se crea cuando el sistema introduce un nuevo trabajo proveniente de una cola de trabajos pendientes, generalmente definidos mediante tarjetas o archivos de control (JCL).*

Formas de Terminación de Procesos

1. *Salida normal (voluntaria): el proceso concluye su tarea satisfactoriamente y solicita al sistema operativo su propia terminación mediante una llamada como exit().*
2. *Salida por error (voluntaria): el propio proceso detecta un error interno (por ejemplo, un archivo requerido no existe) y decide terminar su ejecución de forma controlada.*
3. *Error fatal (involuntaria): el proceso termina de manera abrupta a causa de un error grave provocado por el propio programa, como una división entre cero, una instrucción ilegal o un acceso inválido a memoria (segmentation fault).*
4. *Eliminado por otro proceso (involuntaria): un proceso con los privilegios adecuados —frecuentemente el proceso padre— envía una señal de terminación (por ejemplo, kill() en Unix/Linux) que obliga a otro proceso a finalizar, aun cuando este último no lo haya solicitado.*

2. Diagrama de 5 Estados

El modelo de 5 estados es la representación más utilizada para describir el ciclo de vida de un proceso dentro de un sistema operativo multiprogramado. Cada proceso se encuentra, en todo momento, en uno de los siguientes cinco estados, y solo puede transitar entre ellos siguiendo un conjunto de reglas definidas por el sistema operativo.



Definición de cada Estado

- *Nuevo (New): el proceso está siendo creado; el sistema operativo aún no lo ha admitido formalmente en la cola de procesos listos, generalmente porque está reservando o preparando los recursos necesarios (memoria, estructuras de control, etc.).*
- *Listo (Ready): el proceso ya se encuentra en memoria principal, cuenta con todos los recursos que necesita salvo la CPU, y está a la espera de que el planificador (scheduler) le asigne el procesador.*
- *En ejecución (Running): las instrucciones del proceso se están ejecutando actualmente en la CPU. En un sistema con un solo procesador, únicamente un proceso puede estar en este estado en un instante dado.*
- *Bloqueado / En espera (Waiting o Blocked): el proceso no puede continuar su ejecución porque está esperando que ocurra un evento externo, como la finalización de una operación de entrada/salida, la liberación de un recurso o una señal de otro proceso.*
- *Terminado (Exit): el proceso ha finalizado su ejecución (normal o anormalmente) y el sistema operativo procede a liberar los recursos que tenía asignados.*

Transiciones Válidas entre Estados

- *Nuevo → Listo (admisión): el sistema operativo acepta el proceso, le asigna un Bloque de Control de Proceso (PCB) y lo incorpora a la cola de listos.*
- *Listo → En ejecución (despacho): el planificador de corto plazo (dispatcher) selecciona al proceso y le asigna la CPU.*
- *En ejecución → Listo (interrupción o fin de quantum): el sistema operativo retira la CPU al proceso, ya sea porque terminó su porción de tiempo (quantum) o porque llegó un proceso de mayor prioridad, regresándolo a la cola de listos sin haber terminado.*
- *En ejecución → Bloqueado (espera de E/S o evento): el proceso solicita una operación que no puede completarse de inmediato (por ejemplo, leer un archivo), por lo que cede la CPU y pasa a esperar dicho evento.*
- *Bloqueado → Listo (fin de E/S o evento): una vez que el evento esperado ocurre, el proceso vuelve a estar en condiciones de ejecutarse y regresa a la cola de listos.*
- *En ejecución → Terminado (fin/exit): el proceso concluye su ejecución (de forma normal o por un error) y el sistema operativo libera sus recursos.*

3. Algoritmos de Planificación

a. Definición de Algoritmos de Planificación

Un algoritmo de planificación (scheduling algorithm) es el conjunto de reglas y criterios que utiliza el planificador de corto plazo del sistema operativo para decidir, en cada instante, cuál de los procesos que se encuentran en la cola de listos recibirá la CPU y durante cuánto tiempo la conservará. El objetivo de estos algoritmos es optimizar distintos criterios de desempeño, tales como maximizar el uso de la CPU y el throughput (número de procesos completados por unidad de tiempo), así como minimizar el tiempo de espera, el tiempo de retorno (turnaround) y el tiempo de respuesta, procurando además un reparto equitativo de la CPU entre todos los procesos.

b. Políticas de Planificación

Apropiativas (preemptive / con desalojo): en esta política, el sistema operativo tiene la capacidad de quitarle la CPU a un proceso que se encuentra en ejecución antes de que éste termine voluntariamente su ráfaga, para asignarla a otro proceso. Esto puede suceder, por ejemplo, cuando se agota el quantum de tiempo asignado o cuando llega a la cola un proceso de mayor prioridad. Su principal ventaja es que permite mejores tiempos de respuesta y resulta indispensable en sistemas interactivos y de tiempo real, aunque incrementa la sobrecarga (overhead) debida a los cambios de contexto.

No apropiativas (non-preemptive / sin desalojo): bajo esta política, una vez que un proceso obtiene la CPU la conserva hasta que termina su ráfaga de forma voluntaria, se bloquea esperando una operación de E/S, o finaliza su ejecución; el sistema operativo no puede interrumpirlo para asignar el procesador a otro proceso. Son algoritmos más sencillos de implementar y generan menos cambios de contexto, pero un proceso con una ráfaga muy larga puede retrasar considerablemente a los procesos que esperan detrás de él (efecto convoy).

c. Tabla Comparativa de Algoritmos de Planificación

Algoritmo	Descripción	Política	Ventaja	Desventaja
FCFS	*Los procesos se atienden en el mismo orden en que llegan a la cola de listos (First Come, First Served), como una fila FIFO.*	*No apropiativo*	*Muy simple de entender e implementar; justo respecto al orden de llegada.*	*Genera el efecto convoy: un proceso largo retrasa a los cortos que llegaron después; tiempo de espera promedio alto.*
SJF	*Se selecciona para ejecutarse el proceso con la ráfaga de CPU (siguiente ejecución) más corta entre los disponibles.*	*No apropiativo*	*Minimiza el tiempo de espera promedio entre los algoritmos no apropiativos.*	*Requiere conocer o estimar la duración de la ráfaga de cada proceso; riesgo de inanición para procesos largos.*

Algoritmo	Descripción	Política	Ventaja	Desventaja
SRT	*Versión apropiativa de SJF (Shortest Remaining Time): si llega un proceso nuevo cuya ráfaga es menor al tiempo restante del proceso en ejecución, éste es desalojado de inmediato.*	*Apropiativo*	*Ofrece el tiempo de espera promedio más bajo posible entre todos los algoritmos.*	*Alto overhead por cambios de contexto frecuentes; también requiere estimar duraciones y puede causar inanición.*
RR (Round Robin)	*A cada proceso se le asigna un intervalo fijo de tiempo (quantum); si no concluye en ese lapso, es interrumpido y enviado al final de la cola de listos.*	*Apropiativo*	*Reparto equitativo de la CPU; ideal para sistemas de tiempo compartido; no hay inanición.*	*El rendimiento depende críticamente del tamaño del quantum: uno muy pequeño eleva el overhead y uno muy grande se comporta como FCFS.*
Prioridades	*A cada proceso se le asigna un valor de prioridad; el planificador siempre otorga la CPU al proceso listo con la prioridad más alta.*	*Apropiativo o no apropiativo (según implementación)*	*Permite atender primero a los procesos críticos, del sistema o urgentes.*	*Los procesos de baja prioridad pueden sufrir inanición; se corrige normalmente con la técnica de envejecimiento (aging).*
Colas Múltiples (MLQ)	*Divide la cola de listos en varias colas independientes según el tipo de proceso (por ejemplo, interactivos y por lotes), cada una con su propio algoritmo y una prioridad fija entre colas.*	*Combinada (apropiativa entre colas)*	*Permite tratar de forma diferenciada a procesos con características y necesidades distintas.*	*Es rígida: un proceso no puede cambiar de cola; las colas de menor prioridad pueden sufrir inanición.*
HRRN	*Highest Response Ratio Next. Calcula para cada proceso en espera una razón de respuesta = (tiempo de espera + ráfaga) / ráfaga, y elige siempre al proceso con el valor más alto.*	*No apropiativo*	*Evita la inanición de procesos largos, ya que su razón de respuesta crece mientras más tiempo esperan; equilibra a SJF con equidad.*	*Requiere recalcular constantemente la razón de respuesta de todos los procesos en espera, aumentando la sobrecarga del planificador.*
Colas Multinivel c/ Realimentación (MLFQ)	*Variante de las colas múltiples en la que un proceso sí puede cambiar de cola según su comportamiento (baja de prioridad si consume mucha CPU, o sube si es interactivo).*	*Apropiativo*	*Muy flexible y adaptable; favorece a los procesos cortos e interactivos sin condenar permanentemente a los largos.*	*Es el algoritmo más complejo de configurar e implementar (número de colas, quantum de cada una y criterios de migración).*

4. Elementos de la Tabla de Procesos (BCP)

El Bloque de Control de Proceso (BCP), conocido en inglés como Process Control Block (PCB), es la estructura de datos que el sistema operativo utiliza para almacenar toda la información relevante de cada proceso. Cada vez que ocurre un cambio de contexto, el sistema operativo guarda el estado del proceso saliente y restaura el del proceso entrante utilizando la información contenida en su BCP correspondiente. Entre los elementos principales que integran esta estructura se encuentran:

- *Identificador del proceso (PID): número único que distingue a cada proceso dentro del sistema.*
- *Identificador del proceso padre (PPID): referencia al proceso que creó al proceso actual.*
- *Estado del proceso: indica en cuál de los cinco estados se encuentra actualmente (nuevo, listo, en ejecución, bloqueado o terminado).*
- *Contador de programa (Program Counter): dirección de la siguiente instrucción que debe ejecutarse cuando el proceso vuelva a obtener la CPU.*
- *Registros de CPU: valores de los acumuladores, registros de índice, punteros de pila y códigos de condición al momento de la interrupción.*
- *Información de planificación de la CPU: prioridad del proceso, punteros a las colas de planificación y otros parámetros que utiliza el algoritmo de planificación.*
- *Información de gestión de memoria: tablas de páginas o segmentos, límites y direcciones base de la memoria asignada al proceso.*
- *Información de contabilidad (accounting): tiempo total de CPU consumido, tiempo límite de ejecución, número de proceso y otros datos estadísticos.*
- *Información de estado de entrada/salida: lista de dispositivos de E/S asignados al proceso y lista de archivos que tiene abiertos.*
- *Privilegios e información de seguridad: permisos del proceso, identificador del usuario propietario y credenciales de acceso a recursos protegidos.*

Preguntas de Investigación

1. Describa en qué consisten los algoritmos de planificación No Apropiativos.

Los algoritmos de planificación no apropiativos (non-preemptive) son aquellos en los que, una vez que un proceso obtiene el uso de la CPU, la conserva hasta que él mismo la libera voluntariamente, ya sea porque concluyó su ráfaga de ejecución, porque solicitó una operación de entrada/salida que lo obliga a bloquearse, o porque terminó su ejecución por completo. En este esquema, el sistema operativo no tiene la capacidad de interrumpir a un proceso en ejecución para asignarle la CPU a otro, incluso si llega un proceso de mayor prioridad o con una ráfaga más corta. Ejemplos representativos de esta política son FCFS, SJF (en su forma clásica) y HRRN. Su principal ventaja es la simplicidad de implementación y la baja sobrecarga por cambios de contexto; su principal desventaja es que un proceso con una ráfaga muy larga puede retener la CPU durante mucho tiempo, provocando que los procesos posteriores en la cola de listos esperen más de lo deseable (efecto convoy), lo que puede afectar negativamente el tiempo de respuesta en sistemas interactivos.

2. Explique cómo obtener cada uno de los tiempos solicitados en el punto 9 del programa 3 (Actividad de Aprendizaje 6).

En los ejercicios de planificación de procesos —como los que se resuelven en el programa 3 de la Actividad de Aprendizaje 6— generalmente se pide calcular los siguientes tiempos a partir de dos datos base de cada proceso: su tiempo de llegada (Ta) y su tiempo de ráfaga o servicio (Ts). A partir de la simulación del algoritmo correspondiente (dibujando el diagrama de Gantt) se obtienen los tiempos de finalización, y con ellos el resto de las métricas, de la siguiente manera:

- *Tiempo de finalización (Tf): es el instante en el que el proceso termina por completo su ejecución en la CPU. Se obtiene directamente del diagrama de Gantt generado al aplicar el algoritmo de planificación.*
- *Tiempo de retorno o de servicio (Tr): también llamado turnaround time. Se calcula como Tr = Tf – Ta, es decir, la diferencia entre el tiempo de finalización y el tiempo de llegada del proceso; representa el tiempo total que el proceso permaneció en el sistema.*
- *Tiempo de espera (Te): se obtiene como Te = Tr – Ts, restando al tiempo de retorno la duración de la ráfaga; representa el tiempo que el proceso estuvo en la cola de listos sin recibir la CPU.*
- *Tiempo de respuesta (Tres): se calcula como Tres = Ti – Ta, donde Ti es el instante en que el proceso recibe la CPU por primera vez; mide cuánto tarda el sistema en atender al proceso desde que llegó.*

- *Promedios: finalmente, el tiempo de espera promedio y el tiempo de retorno promedio se obtienen sumando los tiempos individuales de todos los procesos y dividiendo entre el número total de procesos (n), lo que permite comparar la eficiencia de distintos algoritmos de planificación sobre un mismo conjunto de procesos.*

3. ¿Qué significa BCP?

BCP significa Bloque de Control de Proceso, conocido en inglés como Process Control Block (PCB). Es la estructura de datos interna que el sistema operativo crea y mantiene para cada proceso existente en el sistema, y en la cual se almacena toda la información necesaria para administrarlo: su identificador, su estado, el contador de programa, los registros de la CPU, la información de planificación, de gestión de memoria, de contabilidad y de entrada/salida, entre otros datos descritos en la sección anterior. Gracias al BCP, el sistema operativo puede suspender y reanudar procesos (cambio de contexto) sin perder información sobre su ejecución.

Captura del Quiz



Conclusiones

El desarrollo de esta investigación permitió profundizar en uno de los aspectos más importantes de los sistemas operativos: la gestión del ciclo de vida de los procesos y los mecanismos que determinan cómo se reparte el recurso más disputado de cualquier computadora, la unidad central de procesamiento. A lo largo del trabajo se abordaron, de manera progresiva, las formas de creación y terminación de procesos, el modelo de cinco estados con sus transiciones, los algoritmos de planificación con sus políticas apropiativas y no apropiativas, y finalmente la estructura interna que hace posible toda esta administración: el Bloque de Control de Proceso.

En primer lugar, el estudio de las formas de creación y terminación de procesos deja en evidencia que un proceso nunca surge ni desaparece de manera arbitraria: siempre existe un evento disparador —la inicialización del sistema, una llamada de un proceso padre, una solicitud del usuario o el arranque de un trabajo por lotes— y, de la misma manera, siempre existe una causa concreta de terminación, ya sea voluntaria (salida normal o por error controlado) o involuntaria (un error fatal o la intervención de otro proceso). Comprender estas rutas de entrada y salida es fundamental porque son precisamente los puntos donde el sistema operativo reserva y libera los recursos que administra, y un manejo deficiente de estos momentos puede derivar en fugas de memoria, procesos zombis o huérfanos.

En segundo lugar, el diagrama de cinco estados demostró ser el modelo más completo para representar el comportamiento real de un proceso en un entorno multiprogramado, superando al modelo simplificado de tres estados al incorporar explícitamente los estados de Nuevo y Terminado. Cada transición estudiada —admisión, despacho, interrupción, bloqueo y desbloqueo— corresponde a una decisión concreta del sistema operativo, ya sea del planificador de largo plazo (al admitir un proceso), del planificador de corto plazo (al despachar o interrumpir), o del subsistema de entrada/salida (al notificar la finalización de un evento). Esta comprensión resulta indispensable antes de abordar cualquier algoritmo de planificación, ya que dichos algoritmos operan precisamente sobre la transición Listo → En ejecución.

En tercer lugar, el análisis de los algoritmos de planificación —FCFS, SJF, SRT, Round Robin, prioridades, colas múltiples, HRRN y colas multinivel con realimentación— evidencia que no existe un algoritmo universalmente superior, sino que cada uno responde a un equilibrio distinto entre simplicidad, equidad, tiempo de respuesta y riesgo de inanición. Los algoritmos no apropiativos, como FCFS y SJF, destacan por su sencillez, pero son vulnerables al efecto convoy y poco adecuados

para sistemas interactivos. Por su parte, los algoritmos apropiativos, como SRT y Round Robin, sacrifican algo de eficiencia y aumentan la sobrecarga por cambios de contexto a cambio de una mejor capacidad de respuesta, característica indispensable en los sistemas operativos modernos de propósito general y de tiempo compartido. Los algoritmos híbridos o más sofisticados, como HRRN y las colas multinivel con realimentación, muestran cómo la evolución de estas técnicas ha buscado combinar lo mejor de ambos mundos: minimizar tiempos de espera sin sacrificar la equidad ni condenar a la inanición a los procesos de mayor duración.

Finalmente, el estudio del Bloque de Control de Proceso permitió cerrar el círculo conceptual de la investigación: todo lo que un algoritmo de planificación necesita saber sobre un proceso —su estado, su prioridad, su contador de programa, sus registros y sus recursos asignados— se encuentra almacenado precisamente en esta estructura. El BCP es, en este sentido, el soporte de datos que hace posible tanto el modelo de cinco estados como la ejecución de cualquier algoritmo de planificación, confirmando que los distintos temas abordados en este trabajo no son conceptos aislados, sino piezas de un mismo mecanismo integrado que permite a un sistema operativo multiprogramado ejecutar, de manera eficiente y ordenada, decenas o cientos de procesos con recursos limitados.

En conclusión, esta investigación reafirma que la planificación de procesos es, en esencia, un problema de optimización con múltiples objetivos en conflicto (uso de CPU, throughput, tiempo de espera, tiempo de respuesta y equidad), y que el diseño de un sistema operativo siempre implica decidir qué criterios priorizar según el contexto de uso: un sistema por lotes puede privilegiar el throughput con FCFS o SJF, mientras que un sistema interactivo o de tiempo real requiere necesariamente mecanismos apropiativos como Round Robin o colas multinivel con realimentación para garantizar tiempos de respuesta aceptables para el usuario final.

Bibliografía

- *Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10a ed.). Wiley.*
- *Stallings, W. (2018). Operating Systems: Internals and Design Principles (9a ed.). Pearson.*
- *Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4a ed.). Pearson.*
- *Universidad de Valladolid. (s.f.). Tema 5: Planificación de Procesos. https://www.infor.uva.es/~fjgonzalez/apuntes/Tema5.pdf*
- *GeeksforGeeks. (2024). Process Control Block (PCB). https://www.geeksforgeeks.org/process-control-block-pcb/*
- *GeeksforGeeks. (2024). CPU Scheduling in Operating Systems. https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/*